

System Verilog based Function Verification of 11011 sequence detector (overlapping/non-overlapping)

Team Members:

Sangeerthana (2023BEC0044)

Amruth (2023BEC0050)

Faculty Mentor: Dr. Lakshmi NS

March 13, 2026

Contents

1	Overview of Sequence Detector	2
2	Project Workflow	3
3	RTL Design for DUT	4
4	Verification Concepts and Their Implementation	10

1 Overview of Sequence Detector

Our reference is <https://yue-guo.com/2020/01/14/sequence-detector-11011-moore-machine-mealy-machine-overlapping-non-overlapping/>

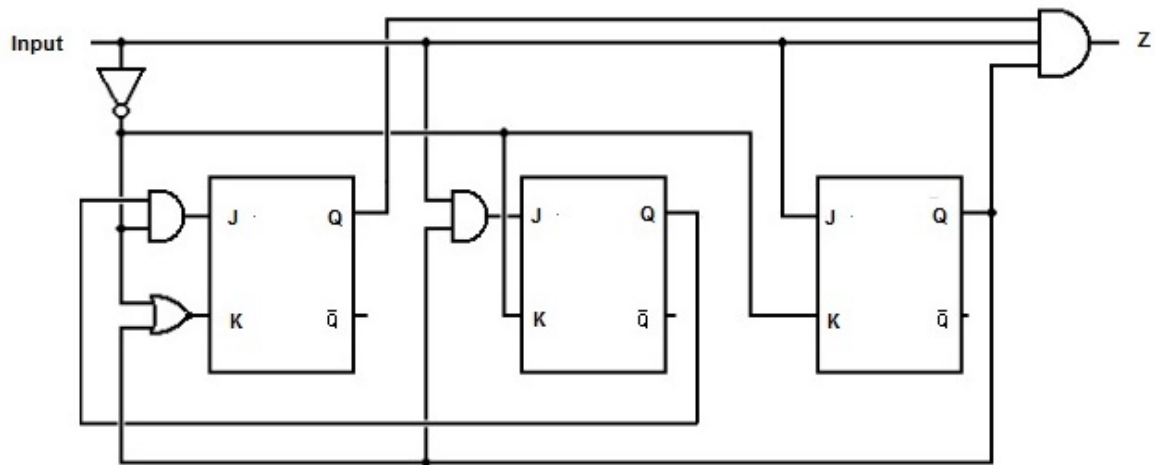
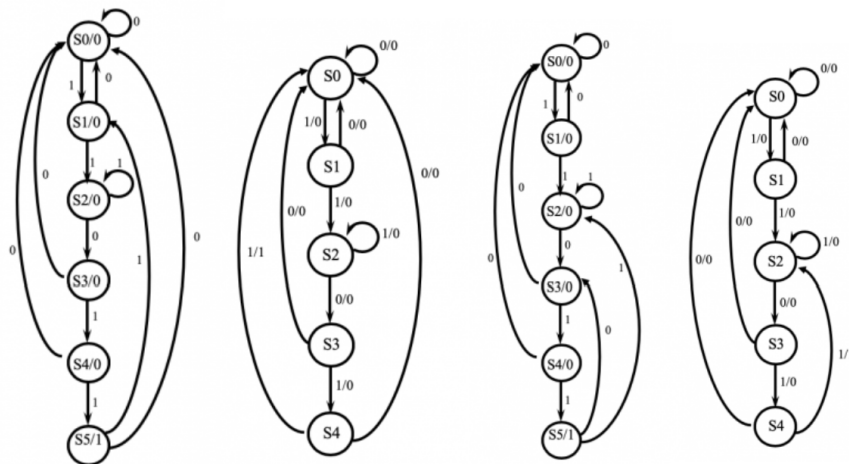


Figure 1: 11011 Sequence Detector Circuit

Below are FSM design for

Moore Machine (Non-Overlapping) , Mealy Machine (Non-Overlapping), Moore Machine (Overlapping) and Mealy Machine (Overlapping)



2 Project Workflow

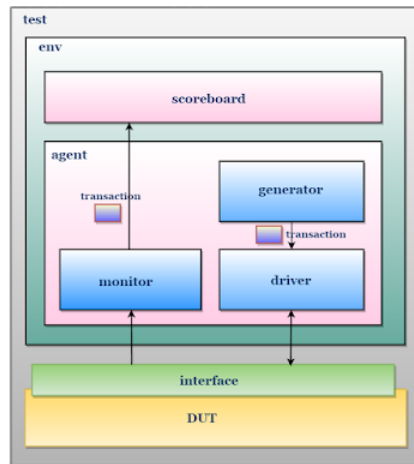


Figure 2: System Verilog Testbench Hierarchy

Project File Structure:

```

testbench_top.sv
|
|--sequence_detector_moore_overlapping.sv
|
|--sequence_detector_moore_non_overlapping.sv
|
|--sequence_detector_mealy_overlapping.sv
|
|--sequence_detector_mealy_non_overlapping.sv
|
|--test.sv
|
|   |--env.sv
|   |
|   |   |--transaction.sv
|   |   |
|   |   |--scoreboard.sv
|   |   |
|   |   |--agent.sv
|   |   |
|   |   |--generator.sv
|   |   |
|   |   |--driver.sv
|   |   |
|   |   |--monitor.sv
  
```

3 RTL Design for DUT

1) Non-Overlapping Moore Sequence Detector of 11011

```
module sd11011_moore(input bit clk,
                    input logic reset,
                    input logic din,
                    output logic dout);

typedef enum logic [2:0] {S0, S1, S2, S3, S4, S5} state_t;
state_t state;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        dout <= 1'b0;
        state <= S0;
    end
    else begin
        case(state)
            S0: begin
                dout <= 1'b0;
                if(din)
                    state <= S1;
            end
            S1: begin
                dout <= 1'b0;
                if(din)
                    state <= S2;
                else
                    state <= S0;
            end
            S2: begin
                dout <= 1'b0;
                if(~din)
                    state <= S3;
            end
            S3: begin
                dout <= 1'b0;
                if(din)
                    state <= S4;
                else
                    state <= S0;
            end
            S4: begin
                dout <= 1'b0;
                if(din)
                    state <= S5;
                else
                    state <= S0;
            end
        endcase
    end
end
```

```

        state <= S0;
    end
S5: begin
    dout <= 1'b1;
    if(din)
        state <= S1;
    else
        state <= S0;
    end
endcase
end
end

```

```
endmodule
```

2) Non-Overlapping Mealy Sequence Detector of 11011

```

module sd11011_mealy(input bit clk,
                    input logic reset,
                    input logic din,
                    output logic dout);

    typedef enum logic [2:0] {S0, S1, S2, S3, S4} state_t;
    state_t state;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            dout <= 1'b0;
            state <= S0;
        end
        else begin
            case(state)
                S0: begin
                    if(din) begin
                        state <= S1;
                        dout <=1'b0;
                    end
                    else
                        dout <=1'b0;
                end
                S1: begin
                    if(din) begin
                        state <= S2;
                        dout <=1'b0;
                    end

```

```

        else begin
            state <= S0;
            dout <=1'b0;
        end
    end
end
S2: begin
    if(~din) begin
        state <= S3;
        dout <=1'b0;
    end
    else begin
        dout <=1'b0;
    end
end
S3: begin
    if(din) begin
        state <= S4;
        dout <=1'b0;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
S4: begin
    if(din) begin
        state <= S0;
        dout <=1'b1;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
endcase
end
end
endmodule

```

3) Overlapping Moore Sequence Detector of 11011

```

module sd11011_moore_over(input bit clk,
                        input logic reset,
                        input logic din,
                        output logic dout);

```

```
typedef enum logic [2:0] {S0, S1, S2, S3, S4, S5} state_t;
state_t state;

always @(posedge clk or posedge reset) begin
    if(reset) begin
        dout <= 1'b0;
        state <= S0;
    end
    else begin
        case(state)
            S0: begin
                dout <= 1'b0;
                if(din)
                    state <= S1;
            end
            S1: begin
                dout <= 1'b0;
                if(din)
                    state <= S2;
                else
                    state <= S0;
            end
            S2: begin
                dout <= 1'b0;
                if(~din)
                    state <= S3;
            end
            S3: begin
                dout <= 1'b0;
                if(din)
                    state <= S4;
                else
                    state <= S0;
            end
            S4: begin
                dout <= 1'b0;
                if(din)
                    state <= S5;
                else
                    state <= S0;
            end
            S5: begin
                dout <= 1'b1;
                if(din)
                    state <= S2;
                else
                    state <= S3;
            end
        endcase
    end
end
```

```

        end
    endcase
end
end

```

```

endmodule

```

4) Overlapping Mealy Sequence Detector of 11011

```

module sd11011_mealy_over(input bit clk,
                        input logic reset,
                        input logic din,
                        output logic dout);

    typedef enum logic [2:0] {S0, S1, S2, S3, S4} state_t;
    state_t state;

    always @(posedge clk or posedge reset) begin
        if(reset) begin
            dout <= 1'b0;
            state <= S0;
        end
        else begin
            case(state)
                S0: begin
                    if(din) begin
                        state <= S1;
                        dout <=1'b0;
                    end
                    else
                        dout <=1'b0;
                end
                S1: begin
                    if(din) begin
                        state <= S2;
                        dout <=1'b0;
                    end
                    else begin
                        state <= S0;
                        dout <=1'b0;
                    end
                end
                S2: begin
                    if(~din) begin
                        state <= S3;

```

```
        dout <=1'b0;
    end
    else begin
        dout <=1'b0;
    end
end
S3: begin
    if(din) begin
        state <= S4;
        dout <=1'b0;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
S4: begin
    if(din) begin
        state <= S2;
        dout <=1'b1;
    end
    else begin
        state <= S0;
        dout <=1'b0;
    end
end
endcase
end
end
endmodule
```

4 Verification Concepts and Their Implementation

1. Constraint Randomization

- (a) Constraint randomization is used in the transaction class to generate random input bits (din). This ensures that many different input patterns are applied to the DUT, increasing the probability of detecting the sequence 11011, including overlapping cases.

- (b)


```

      rand bit din;
      constraint c { din inside {0,1}; }
      bit detect;
      endclass
      
```

- (c) Design space exploration basically

2. Transaction Class

- (a) A transaction class represents a unit of stimulus sent to the DUT. It stores the input bit and may also include the expected output.

- (b)


```

      class transaction;
      rand bit din;
      bit detect;
      endclass
      
```

- (c) Stimulus becomes an object and the communication between verification components

3. Mailbox Communication

- (a) A mailbox is created in the testbench to transfer transactions between components such as the generator and driver.

- (b)


```

      class transaction;
      rand bit din;
      bit detect;
      endclass
      
```

- (c) gen2drv and mon2scb are two typical mailbox placements in the code, and the .put() and .get() methods help send and retrieve the stimuli and the DUT's output captured via the virtual interface. Mailbox typically helps sync the concurrent transfers between gen2drv and mon2scb.

4. Generator Class

- (a) The generator creates transaction objects, randomizes the input bit, and sends the transaction to the driver through the mailbox.

5. Driver Class with Virtual Interface

- (a) The driver receives transactions from the mailbox and applies the stimulus to the DUT through a virtual interface connected to the DUT signals.

- i. Receive transaction from mailbox
- ii. Wait for clock edge
- iii. Drive din signal to DUT via virtual interface

6. Parallel Execution using fork-join

- (a) The generator and driver operate concurrently using fork-join to simulate a multithreaded testbench environment.

(b)

```
fork
    generator.run()
    driver.run()
join
```

7. Monitor

- (a) The monitor passively observes DUT signals and converts them into transactions. It captures the input bit stream and the detection output and forwards this information to the scoreboard.

8. Scoreboard

- (a) The scoreboard compares the DUT output with the expected result generated by a reference model.
- (b) the scoreboard maintains a 5-bit shift register, checks if the sequence equals 11011, compares expected detection with DUT detection

9. Reference Model

- (a) The reference model represents the correct behavior of the sequence detector. It predicts the expected output based on the input stream.

(b)

```
shift_reg = {shift_reg[3:0], din}

if shift_reg == 11011
    expected_detect = 1
```

10. Immediate Assertions and Concurrent Assertions

- (a) Immediate assertions are used inside procedural blocks to check conditions during simulation. like `assert(detect == 1 when sequence == 11011);`
- (b) Concurrent assertions verify temporal relationships across clock cycles. They ensure that the detection signal is asserted only when the correct sequence appears in the input stream.

11. Functional Coverage

- (a) Functional coverage is implemented to measure how thoroughly the input scenarios have been exercised during simulation.
- (b) Coverage items include:

- i. input bit values
- ii. sequence occurrence
- iii. detection events

12. Covergroups and Coverpoints

- (a) A covergroup is defined to collect coverage information. Coverpoints track the values of signals such as the input bit and detection signal.
 - i. coverpoint din
 - ii. coverpoint detect

13. Cross Coverage

- (a) Cross coverage analyzes combinations of signals, such as the relationship between the input bit and the detection signal.
 - i. cross din, detect

14. Cross Coverage

- (a) Cross coverage analyzes combinations of signals, such as the relationship between the input bit and the detection signal.
 - i. cross din, detect